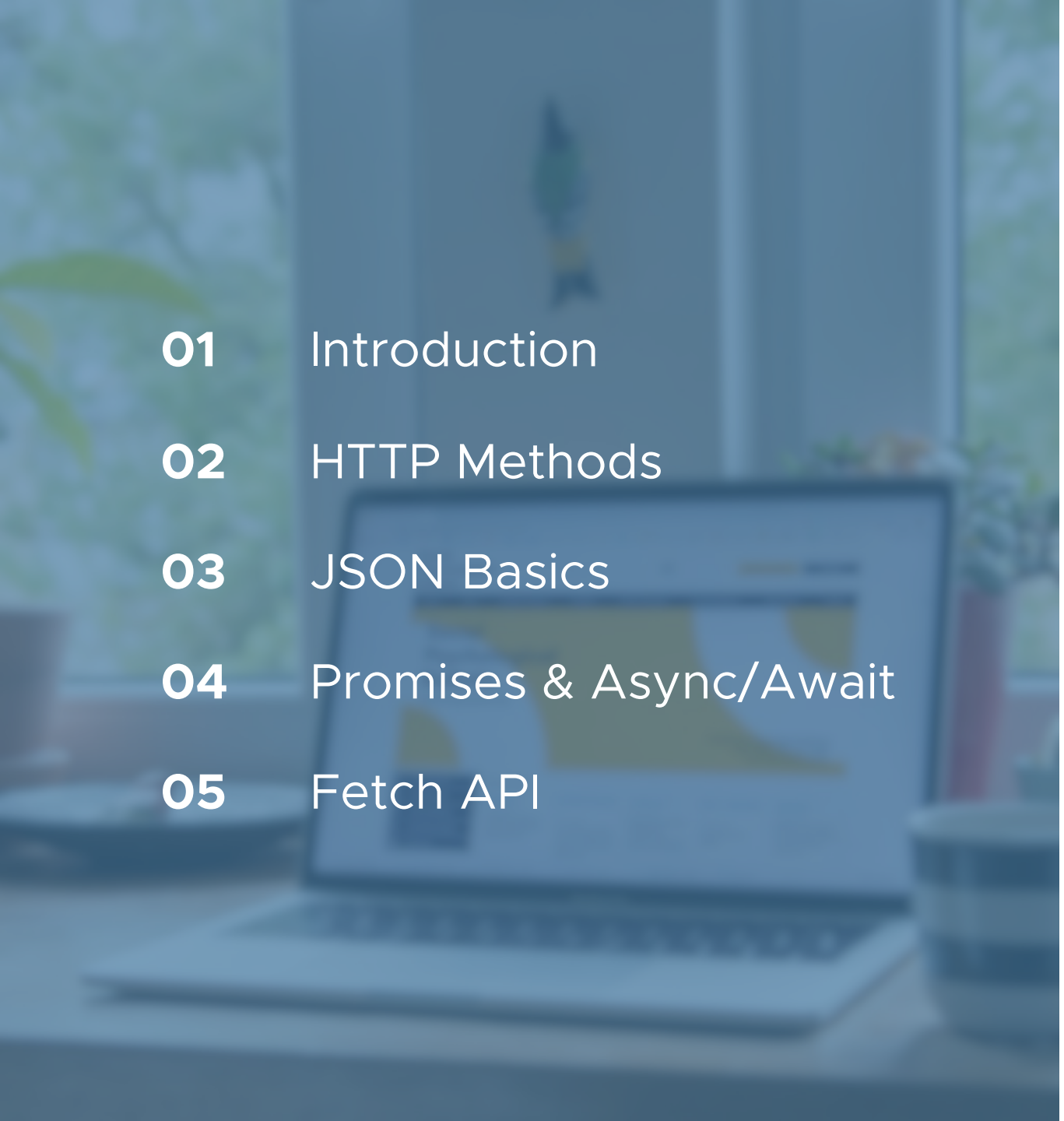
Full Stack Development Course

Eng. Elias Al Zaghrini

Application Program Interfaces (APIs)

Chapter 5



01 Introduction

02 HTTP Methods

03 JSON Basics

04 Promises & Async/Await

05 Fetch API

Full Stack Development Course

Eng. Elias Al Zaghrini

Application Program Interfaces (APIs)

Chapter 5

Introduction

What is an API?

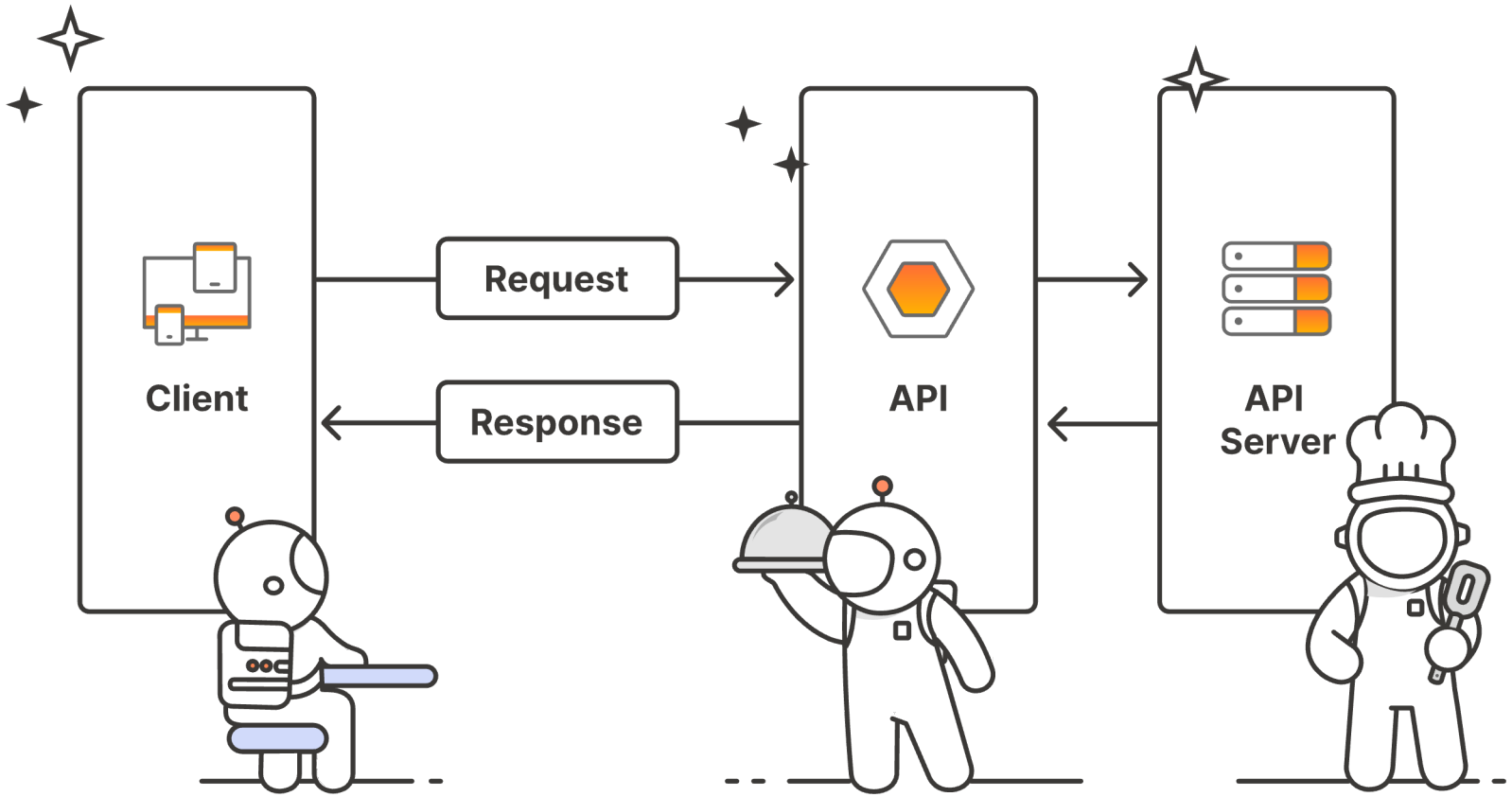
API = Application Programming Interface

- A set of rules that allows different software components to communicate.
- In web development, APIs let your frontend app talk to servers or external services.
- Examples: Weather API, Google Maps API, Payment Gateways

Introduction

What is an API?

API = Application Programming Interface



Introduction

What is a REST API?

REST = Representational State Transfer

A common architecture for designing networked applications

Works over HTTP using:

- GET: Retrieve data
- POST: Send new data
- PUT: Update data
- DELETE: Remove data Uses URLs to identify resources (e.g., /users/1)

HTTP Methods

Request components

An API Request is what your app sends to a server.

1. Endpoint (URL):

- The address of the resource, e.g., https://api.example.com/users

2. Method:

- GET: Retrieve data
- POST: Send new data
- PUT: Update data
- DELETE: Remove data

HTTP Methods

Request components

3. Headers:

- Extra info like content type or authentication token.

4. Body (Optional):

- Sent with POST, PUT, etc., usually in JSON format.
- Example:

```
{
  "name": "John",
  "email": "john@example.com"
}
```

HTTP Methods

Response components

An API Response is the server's reply to your request.

1. Status Code: Indicates success or failure

- 200 OK: Success
- 201 Created: Resource added
- 404 Not Found: Wrong URL or missing data
- 500 Server Error: Internal failure

2. Headers:

- Metadata like content type, date, rate limits, etc.

HTTP Methods

Response components

3. Body:

Contains the actual response data (usually in JSON)

```
{
  "id": 123,
  "name": "John",
  "registered": true
}
```

HTTP Methods

Tools

Tools to Inspect Responses:

- Browser DevTools → Network Tab
- Postman
- Console logs in JavaScript

JSON Basics

Introduction to JSON

- Most web APIs return data in JSON format
- JSON: JavaScript Object Notation
- Human-readable, key-value structure
- Parse JSON in JS: `JSON.parse() / JSON.stringify()`

```
{
  "name": "John",
  "age": 25,
  "isMember": true
}
```

JSON Basics

Introduction to JSON

```
{
  "name": "Jane Doe",
  "age": 30,
  "skills": ["HTML", "CSS", "JavaScript", "React"],
  "projects": [
    {
      "title": "Personal Portfolio",
      "technologies": ["HTML", "CSS", "JavaScript"]
    },
    {
      "title": "E-commerce App",
      "technologies": ["React", "Node.js", "MongoDB"]
    }
  ],
  "contact": {
    "email": "jane.doe@example.com",
    "linkedin": "https://linkedin.com/in/janedoe"
  }
}
```

JSON Basics

Practice

Practice

Postman

JSON Basics

Practice

Endpoint: `https://api.api-ninjas.com/v1/exercises`

Method: `GET`

Header: `X-Api-Key` : `xz6c+hkUuuC+7S/oqjDECw==KZoYBB9STiQdGsnb`

Parameters: `muscle` : `e.g.: biceps, abdominals, back, …`

`difficulty` `e.g.: beginner, intermediate, …`

Promises & Async/Await

What is a Promise?

A Promise:

- An object used to manage asynchronous results
- Has a `then()` method that lets you attach functions to execute `onSuccess` or `onError`
- Allows you to build chains of asynchronous results.

Promises are easier to use than to define...

Promises & Async/Await

Syntax

```
function onStreamProcessed(text) {
  console.log(text);
}

function onResponse(response) {
  return response.text();
}

fetch('images.txt')
    .then(onResponse)
    .then(onStreamProcessed)
    .catch(error);
```

Promises & Async/Await

What is a Promise?

- A Promise represents a value that will be available in the future.
- Used to handle asynchronous operations like API calls.

```
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

Promises & Async/Await

Async/Await (Cleaner Promises)

- Easier to read and write

```
async function getData() {
  try {
    const res = await fetch(url);
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}
```

The Fetch API

What is it?

- Built-in JS method for making HTTP requests
- `fetch()`: Function to load resources in JavaScript

```
fetch(pathToResource)
    .then(onResponse)
    .then(onResourceReady);
```

- `onResponse`: Return response.text() from this function to get the resource as a string in onResourceReady
- `onResourceReady`: Gets the resource as a parameter when it's ready

The Fetch API

What is it?

```
function onTextReady(text) {
  // do something with text
}

function onResponse(response) {
  return response.text();
}

fetch('images.txt')
    .then(onResponse)
    .then(onTextReady);
```

The Fetch API

What is it?

- Supports GET, POST, PUT, DELETE
- Example

```
fetch('https://api.example.com/users')
  .then(res => res.json())
  .then(data => console.log(data));
```

The Fetch API

Form submission

- Capture form input and send via fetch()
- Can send as JSON too

```
document.querySelector('form').addEventListener('submit', async (e) => {
  e.preventDefault();
  const formData = new FormData(e.target);
  const res = await fetch('/submit', {
    method: 'POST',
    body: formData
  });
});
```

The Fetch API

Querying REST APIs

- APIs often allow query parameters to filter or modify the response.

```
https://api.example.com/products?category=books&sort=price_asc&page=2
```

- `?` starts the query string
- `&` separates multiple parameters
- Common use cases:
    - Filtering (`category=books`)
    - Sorting (`sort=price_asc`)
    - Pagination (`page=2`)

The Fetch API

Querying REST APIs

JavaScript Example

```
const url =
'https://api.example.com/products?category=books&sort=price_asc';
fetch(url)
  .then(res => res.json())
  .then(data => console.log(data));
```

The Fetch API

Retrieve JSON data

Let's say we had a file that contained a list of albums.

- Each album has:
    - Title
    - Year
    - URL to album image
- We want to display each album in chronological order.

The Fetch API

Retrieve JSON data

```
Kalamantina
2025
https://i1.sndcdn.com/artworks-Hb9dmi8AmidLFTST-LIrDLA-t500x500.jpg

Ana min
2025
https://images.genius.com/115f08f5db74d0de1dbd4fe3ede1caaf.1000x1000x1.png

Jinak
2025
https://i1.sndcdn.com/artworks-VO6eGoQh0RWW-0-t500x500.jpg
```

The Fetch API

Retrieve JSON data

```
{
  "albums": [
    {
      "name": ”Kalamantina",
      "year": 2025,
      "url": "https://i1.sndcdn.com/AmidLFTST-LIrDLA-t500x500.jpg"
    },
    {
      "name": ”Ana min",
      "year": 2025,
      "url": "https://images.genius.com/115f08f5db74d0d.1000x1000x1.png"
    },
…
```

The Fetch API

Retrieve JSON data

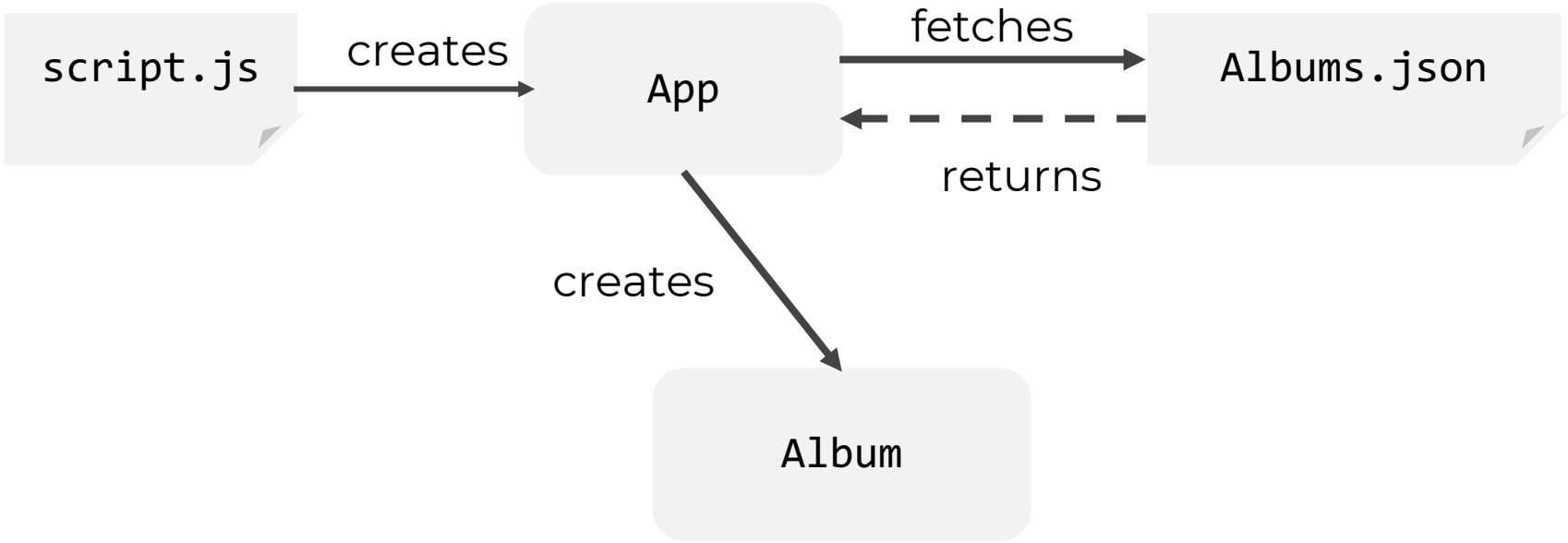
- Since we're using JSON, we don't have to manually convert the response strings to a JavaScript object:
- JavaScript has built-in support to convert a JSON string into a JavaScript object.

```
function onJsonReady(json) {
  const albums = json.albums;
  ...
}
```

The Fetch API

With classes

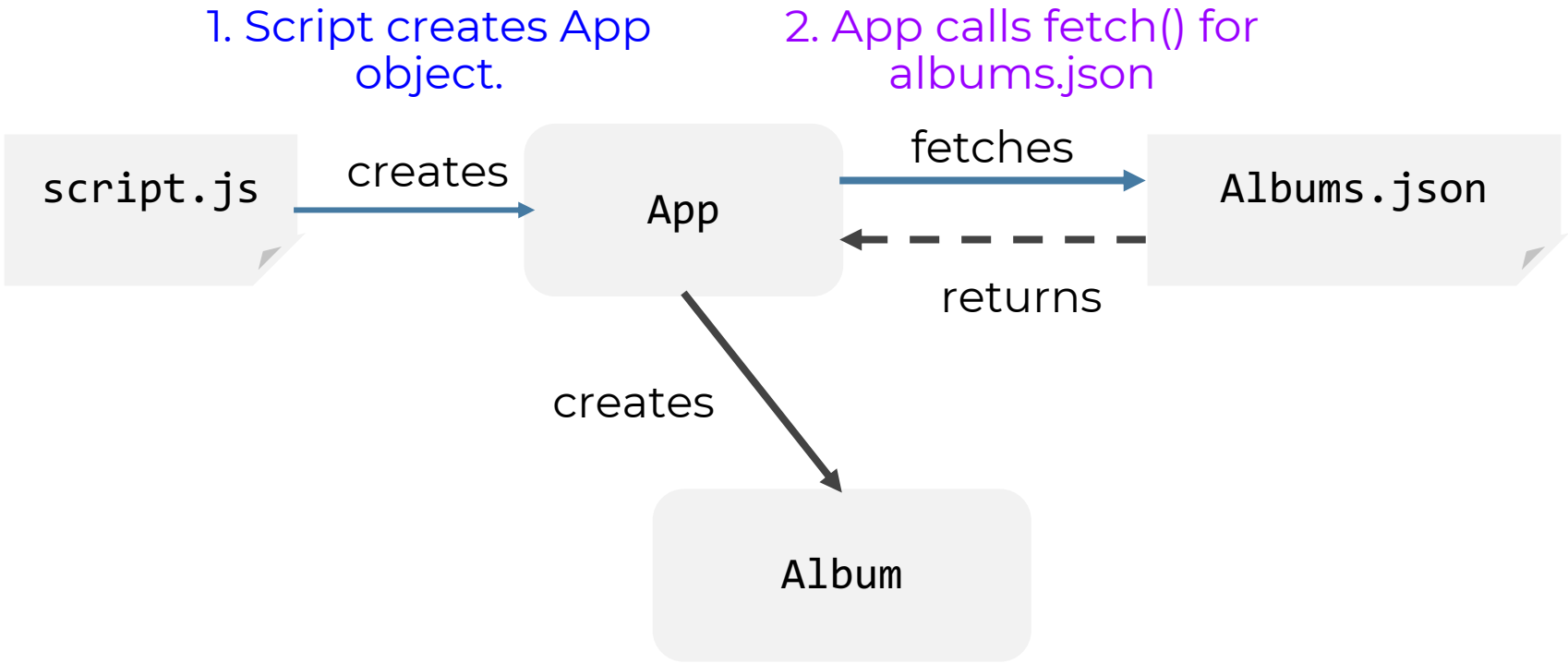
The class diagram is going to look something like this:



The Fetch API

With classes

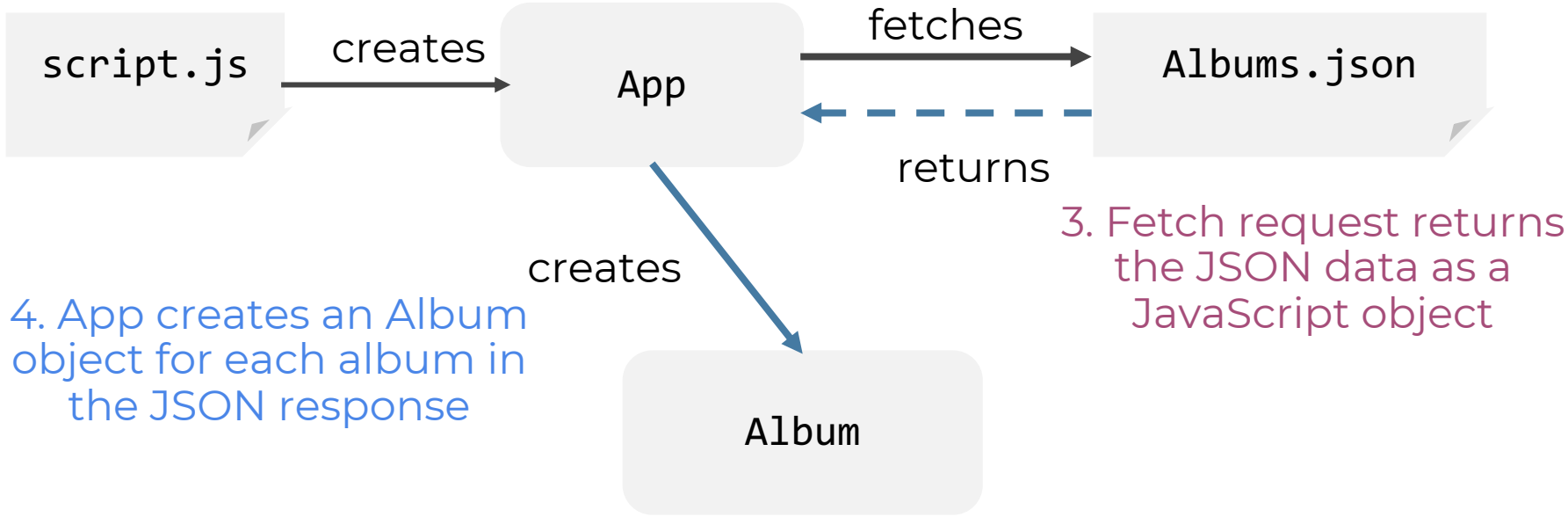
The class diagram is going to look something like this:



The Fetch API

With classes

The class diagram is going to look something like this:



The Fetch API

Example

Example 1

Lebanese Songs

The Fetch API

Example

Example 2

Fill a form

Application Program Interfaces (APIs)

Any questions?